Working with Data

Session 1 — What is this data?

Block 2 — One table, one question: SQL on DuckDB

This block in one sentence

A number is a sentence before it is a query: *a measure, over a population, after a filter.*

SQL is that sentence written precisely enough for a machine to answer it and for a colleague to check it. `NULL` is waiting at every clause.

Agenda

1. The method: build one clause at a time
2. Why SQL: 525,461 lines, watched
3. The map: the order a query runs
4. The number as a sentence · the counts · the other aggregates · `NULL` in them
5. `GROUP BY`, its rule, the `NULL` group
6. The sum identity: the numbers tie out
7. A calculated column · dates · text
8. Lab 2 — on the other file

The method: build one clause at a time

1. **The sentence**: the measure, the population, the filter.
2. `FROM` + `LIMIT 10`: look at ten rows. Are they the thing?
3. `WHERE`: **count the rows** it keeps. The number you expected?
4. `GROUP BY`: **say how many groups** before you run it.
5. The aggregates. Then `ORDER BY`. Remove the `LIMIT`.
6. **The check**: compute the number a second way. The two must tie out.

Why SQL — 525,461 lines, watched

```
$ uv run python - <<'EOF'
import time, duckdb
con = duckdb.connect()
t = time.perf_counter()
lines = con.sql("SELECT COUNT(*) FROM 'data/raw/online_retail.parquet'").fetchone()[0]
by_country = con.sql("""
    SELECT Country, SUM(Quantity * Price) AS revenue
    FROM 'data/raw/online_retail.parquet'
    GROUP BY Country
    ORDER BY revenue DESC
""").df()
print(f"{lines:,} lines, {len(by_country)} countries, {time.perf_counter() - t:.3f} seconds")
EOF
525,461 lines, 40 countries, 0.106 seconds
```

captured on the build machine, 30 September 2026

A query can be read, rerun, and checked. The file is the table: `FROM` names it, nothing is loaded.

The lecture file: one row is one country in one year

Watch (cell 1 of `lecture.ipynb`): Block 1's key test, on the lecture file.

cell 1

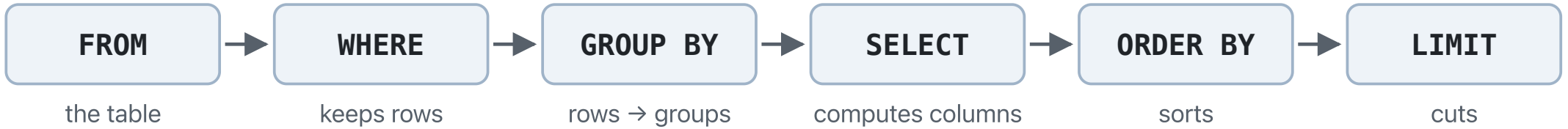
```
con.sql("""
    SELECT COUNT(*)                          AS n_rows,
           COUNT(DISTINCT (country, year)) AS n_keys,
           COUNT(DISTINCT country)         AS countries
    FROM 'data/raw/countries.csv'
""").df()
```

	n_rows	n_keys	countries
0	7665	7665	219

219 countries, in the file's sense, 1990 to 2024. One row per country per year?

The map: the order a query runs

The order it runs



The order you write it: `SELECT · FROM · WHERE · GROUP BY · ORDER BY · LIMIT`

Each clause gets the result of the one before it. Nothing can use what has not been made yet.

You write `SELECT` first. It runs **fourth**.

The sentence first, then the query

Total CO_2 emitted by countries in 2024 is **the sum of** `co2` (the measure) **over the country rows** (the population) **of 2024** (the filter).

In plain English, a sentence says three things: **what the number measures, which rows it comes from, and which rows the query left out.**

Predict, then watch (cell 2):

```
con.sql("""
    SELECT SUM(co2) AS total_co2
    FROM 'data/raw/countries.csv'
    WHERE year = 2024
""").df()
```

Predict: how many rows come back? One per country, or one in all?

The sentence first, then the query — what it printed

cell 2

```
con.sql("""
    SELECT SUM(co2) AS total_co2
    FROM 'data/raw/countries.csv'
    WHERE year = 2024
""").df()
```

	total_co2
0	37406.117

One row, not one per country: an aggregate with no `GROUP BY` collapses the whole filtered table into a single row.

The three counts, and what `NULL` does to them

Predict, then watch (cell 3). Predict: are `n_rows` and `n_co2` equal?

```
con.sql("""
    SELECT COUNT(*)                 AS n_rows,
           COUNT(co2)               AS n_co2,
           COUNT(DISTINCT country) AS n_countries
    FROM 'data/raw/countries.csv'
    WHERE year = 2024
""").df()
```

The three counts, and what `NULL` does to them — what it printed

cell 3

```
con.sql("""
    SELECT COUNT(*)                 AS n_rows,
           COUNT(co2)               AS n_co2,
           COUNT(DISTINCT country) AS n_countries
    FROM 'data/raw/countries.csv'
    WHERE year = 2024
""").df()
```

	n_rows	n_co2	n_countries
0	219	216	219

`COUNT(*)` counts rows. `COUNT(co2)` counts the rows where `co2` is not `NULL`. `COUNT(DISTINCT country)` counts the different values.

Rows are not things

Predict, then watch (cell 4): the same two counts, on every year.

```
con.sql("""
    SELECT COUNT(*)                 AS n_rows,
           COUNT(DISTINCT country) AS n_countries
    FROM 'data/raw/countries.csv'
""").df()
```

Predict: are the two numbers equal this time?

Rows are not things — what it printed

cell 4

```
con.sql("""
    SELECT COUNT(*)                 AS n_rows,
           COUNT(DISTINCT country) AS n_countries
    FROM 'data/raw/countries.csv'
""").df()
```

	n_rows	n_countries
0	7665	219

One row is one country **in one year**. `COUNT(*)` counted rows. `COUNT(DISTINCT country)` counted countries. **Say what one row is before you count.**

The other four aggregates — watch

Watch (cell 5): `SUM`, `AVG`, `MIN`, `MAX`, on 2024.

cell 5

```
con.sql("""
    SELECT SUM(co2) AS total_co2,
           AVG(co2) AS avg_co2,
           MIN(co2) AS min_co2,
           MAX(co2) AS max_co2
    FROM 'data/raw/countries.csv'
    WHERE year = 2024
""").df()
```

	total_co2	avg_co2	min_co2	max_co2
0	37406.117	173.176468	0.0	12289.037

`AVG` divides by the **values that exist**: 216, not 219.

`COALESCE`: a missing value becomes a zero, and a zero is a claim

Watch (cell 6). `COALESCE(gdp, 0)` returns the first of its values that is not `NULL`: `gdp` where there is one, otherwise `0`.

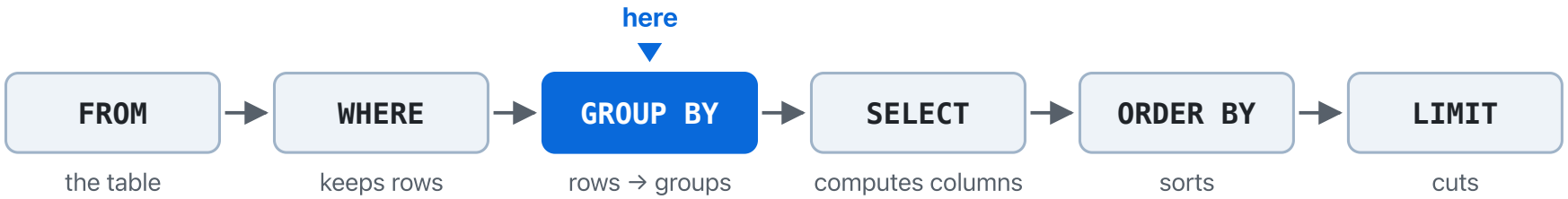
cell 6

```
con.sql("""
    SELECT country, gdp, COALESCE(gdp, 0) AS gdp_or_zero
    FROM 'data/raw/countries.csv'
    WHERE year = 2022 AND country IN ('France', 'Germany', 'Monaco')
    ORDER BY country
""").df()
```

	country	gdp	gdp_or_zero
0	France	2671568767031	2671568767031
1	Germany	3909612753939	3909612753939
2	Monaco	<NA>	0

`GROUP BY`: one row per year

The order it runs



The order you write it: `SELECT · FROM · WHERE · GROUP BY · ORDER BY · LIMIT`

One row per value of the grouping column. Every other column must be aggregated.

Watch (cell 7). One row per year.

`GROUP BY` — what it printed

cell 7

```
con.sql("""
    SELECT year, SUM(co2) AS total_co2
    FROM 'data/raw/countries.csv'
    GROUP BY year
    ORDER BY year
""").df()
```

	year	total_co2
0	1990	22184.276
1	1991	22662.918
2	1992	21966.603
3	1993	22192.438
4	1994	22389.768
5	1995	22899.373
6	1996	23564.971
7	1997	23677.035
8	1998	23549.459
9	1999	24064.355
10	2000	24727.963
11	2001	24915.493

	year	total_co2
12	2002	25468.479
13	2003	26823.432
14	2004	27706.560
15	2005	28659.728
16	2006	29603.555
17	2007	30450.998
18	2008	30991.476
19	2009	30498.959
20	2010	32245.651
21	2011	33373.932
22	2012	33860.648
23	2013	34165.627

	year	total_co2
24	2014	34349.848
25	2015	34250.914
26	2016	34223.754
27	2017	34773.495
28	2018	35497.626
29	2019	35846.991
30	2020	34318.680
31	2021	35987.784
32	2022	36562.105
33	2023	37011.294
34	2024	37406.117

The rule `GROUP BY` enforces

Predict, then watch (cell 8a; it is wrong on purpose). Predict: does DuckDB pick a country, or refuse?

```
SELECT year, country, SUM(co2) FROM 'data/raw/countries.csv' GROUP BY year
```

`year` is the **identifier** of each group. `country` is a **label**: a column that is neither grouped nor aggregated.

The rule `GROUP BY` enforces — what it printed

cell 8a

```
# The try/except prints DuckDB's refusal instead of stopping the notebook.
try:
    con.sql("""
        SELECT year, country, SUM(co2)
        FROM 'data/raw/countries.csv'
        GROUP BY year
    """).df()
except duckdb.Error as e:
    print(e)
```

```
Binder Error: column "country" must appear in the GROUP BY clause or must be
part of an aggregate function.
Either add it to the GROUP BY list, or use "ANY_VALUE(country)" if the exact
value of "country" is not important.
```

Every column in `SELECT` is either in the `GROUP BY` or inside an aggregate. `year` is grouped, `SUM(co2)` is aggregated, `country` is neither: refused.

The fix: put the column in the `GROUP BY`

Watch (cell 8b): the identifier and its label, grouped together.

cell 8b

```
# One row per (iso_code, country): the identifier and its label, grouped together.
con.sql("""
    SELECT iso_code, country, SUM(co2) AS co2_since_1990
    FROM 'data/raw/countries.csv'
    GROUP BY iso_code, country
    ORDER BY co2_since_1990 DESC
    LIMIT 5
""").df()
```

	iso_code	country	co2_since_1990
0	CHN	China	244175.556
1	USA	United States	192495.593
2	RUS	Russia	58797.248
3	IND	India	55646.011
4	JPN	Japan	41902.295

The `NULL` group

Predict, then watch (cell 9). Predict: how many rows — two, or three?

```
con.sql("""
    SELECT gdp > 1000000000000 AS big_economy, COUNT(*) AS countries, SUM(co2) AS co2
    FROM 'data/raw/countries.csv'
    WHERE year = 2022
    GROUP BY big_economy
    ORDER BY big_economy
""").df()
```

The `NULL` group — what it printed

cell 9

```
con.sql("""
    SELECT gdp > 1000000000000 AS big_economy, COUNT(*) AS countries, SUM(co2) AS co2
    FROM 'data/raw/countries.csv'
    WHERE year = 2022
    GROUP BY big_economy
    ORDER BY big_economy
""").df()
```

	big_economy	countries	co2
0	False	138	5611.665
1	True	26	30866.045
2	<NA>	55	84.395

Three rows. The `<NA>` row is the 55 countries whose `gdp` is `NULL`: *is it above a trillion?* is neither true nor false for them, so they are a group of their own. A `WHERE gdp > 1e12` would keep 26; a `WHERE gdp <= 1e12` would keep 138; the 55 would be in **neither**.

The sum identity: the numbers tie out

Predict, then watch (cell 10). Predict: do the three groups, added up, equal the total?

Emissions are **additive**: every breakdown adds up to the total, computed by other code, or one of them is wrong. Finance says the numbers *reconcile*, or *tie out*. This course says **identity**: the equation a number must satisfy.

Compare sums with a tolerance, never `==`. Two correct sums of the same numbers can differ in the tenth decimal. Money: to the penny, `abs(a - b) < 0.005`.

Means and ratios have their own identities. An average does not add up. A ratio's top and bottom are reconciled separately, on the same rows.

The sum identity: the numbers tie out — what it printed

cell 10

```
total = con.sql("SELECT SUM(co2) FROM 'data/raw/countries.csv' WHERE year = 2022").fetchone()[0]
by_group = con.sql("""
    SELECT SUM(co2) FROM (
        SELECT gdp > 1000000000000 AS big_economy, SUM(co2) AS co2
        FROM 'data/raw/countries.csv'
        WHERE year = 2022
        GROUP BY big_economy
    )
""").fetchone()[0]
print(f"Total, 2022:                    {total:10,.3f} Mt")
print(f"The three groups, added up:     {by_group:10,.3f} Mt")
print("equal with == ?", total == by_group, "   within 0.0005 Mt ?", abs(total - by_group) < 0.0005)
```

```
Total, 2022:                    36,562.105 Mt
The three groups, added up:     36,562.105 Mt
equal with == ? False    within 0.0005 Mt ? True
```

The same number, twice. `==` says **False**; within half a thousandth says **True**. The second query is a **subquery**: a query inside a query, `SELECT SUM(…) FROM ( … GROUP BY … )`.

A calculated column, inside the aggregate

Watch (cell 11): tonnes per person, by year. `co2` is in million tonnes and `population` in persons, so `co2 * 1000000 / population`.

cell 11

```
con.sql("""
    SELECT year,
           ROUND(SUM(co2 * 1000000) / SUM(population), 2) AS tonnes_per_person
    FROM 'data/raw/countries.csv'
    WHERE year >= 2020 AND co2 IS NOT NULL AND population IS NOT NULL
    GROUP BY year
    ORDER BY year
""").df()
```

	year	tonnes_per_person
0	2020	4.36
1	2021	4.53
2	2022	4.56
3	2023	4.58
4	2024	4.59

The expression goes **inside** the `SUM`; `AS` names the result; `ROUND(…, 2)` is for display.

Dates, the minimum

Watch (cell 12). One timestamp, three ways:

cell 12

```
con.sql("""
    SELECT date_trunc('month', TIMESTAMP '2010-03-17 14:05:00') AS month_start,
           year(TIMESTAMP '2010-03-17 14:05:00')                AS the_year,
           strftime(TIMESTAMP '2010-03-17 14:05:00', '%Y-%m')   AS label
""").df()
```

	month_start	the_year	label
0	2010-03-01	2010	2010-03

`date_trunc` gives **the start of its month**, a value you can `GROUP BY`. `year` gives the number. `strftime` gives text, for a label.

Text, the minimum: `left`

Watch (cell 13). `left(country, 1)` is the first character of the name; a census can group by it.

cell 13

```
con.sql("""
    SELECT left(country, 1) AS initial, COUNT(*) AS countries
    FROM 'data/raw/countries.csv'
    WHERE year = 2024
    GROUP BY initial
    ORDER BY countries DESC
    LIMIT 5
""").df()
```

	initial	countries
0	S	29
1	B	20
2	C	20
3	M	20
4	A	14

What you just saw, as one idea

A filter that sounds right can throw away rows the sentence keeps.
`WHERE gdp > 1e12` keeps 26 countries and `WHERE gdp <= 1e12` keeps 138; the 55 with no GDP figure are in neither. `GROUP BY` showed them; `WHERE` would not have.

A ratio can be built from two different populations.
`SUM(gdp) / COUNT(*)` is a top from 164 countries over a bottom of 219: 595 billion, the same wrong number as counting the missing as zero.

No error in either. The sentence says what the number counts; the query must do all of it, and only it. **The check finds them.**

Git thread — the message names the cause

```
git commit -m "Keep countries with no GDP figure in the breakdown: gdp <= 1e12 dropped 55 NULL rows"
```

not `git commit -m "fix"`. Commit after each fix. Six months from now, `git log --oneline` is the only diagnosis note anyone will read.

AI thread — the assistant does not know your data

Ask an assistant: *"Average GDP per country in 2022, from this table"*, and paste `DESCRIBE`. It will write `AVG(gdp)`. Then ask it: *"over how many countries?"*

It cannot know. `COUNT(gdp)` against `COUNT(*)`: 164 against 219.

The assistant does not know your data's conventions unless you paste them. The census is how you learn them yourself.

5-minute stand-and-stretch — start the Lab 2 clone first

In your terminal (Git Bash on Windows, Terminal on macOS), in the folder where you keep course work:

```
git clone https://github.com/earino/ecbs5294-lab02-one-table.git
cd ecbs5294-lab02-one-table
uv sync
```

Start it, then take your break: it downloads while you are away.

Lab 2 — The revenue the brief did not ask for

The file: `data/raw/online_retail.parquet` — the shop's invoice lines. **The brief:** finance's definitions of every number, in the README and the notebook.

The symptom: the report runs without an error and says 2010 revenue was **£8,739,637.52** and the average identified customer brought in **£2,052.52**. Its own per-customer table does not average to £2,052.52. And every number has a sentence in the brief.

First minute: the clone finished? Open the folder in VS Code, open `notebooks/report.ipynb`, pick the `.venv` kernel, run the first cell: it prints the Lab 2 folder.

Your work, in `report.ipynb`, top to bottom. **A:** run the three supplied inspections and paste what they show into your note. **B:** the brief's filter, written once as a view `sales_2010`, then the census again on it. **C:** revenue by month, from an empty cell. **D:** the five countries with the most invoices, from an empty cell. **E:** the check: your view against the raw file, your months against your view, both to the penny. Then **`DIAGNOSIS.md`**. After the note, if there is time: **F**, the per-customer figure (also HW1 question 7). Neighbour at minute 33. Checkpoint on Moodle. The syntax is on one page: `REFERENCE-sql.md`; the full reference, with a task index, is on the course site.

The last ten minutes — explain it to a neighbour

One of you explains, about a minute, pointing at the screen: **what was wrong · why · the query that proved it · what you changed · how you know it is right.** The other asks:

1. **Show me the query that proves it.**
2. **Why was it wrong, not just where?**
3. **A colleague says your view's filter is the long way round: `WHERE Quantity > 0` *does the same job*. Same lines? Same revenue?**

Then swap. Unsure, or you disagree? **Hands up.** Then the checkpoint on Moodle.

Three kinds of check

Every number has a check its kind permits. The homework names the kind under each question.

The number is…	The check
a sum	split it into groups, and show they add back up to it
a ranked list	count the rows, and reach the top row a second way
a ratio	check the top and the bottom separately, on the same lines

If no query would catch the error: write how the number could be wrong, and what you would see if it were.

Homework 1 assigned — One table, the weekly numbers

The head of sales wants **seven numbers** from the same file; two more are **stretch**, not graded. The brief carries Lab 2's definitions and a few more. About **5 hours** for the seven. Due **Saturday 10 October, 23:59**, on Moodle.

The starter is `earino/ecbs5294-hw01-one-table` on GitHub; its README has the clone line, for home. It opens with a worked example.

Five parts under every question, in the notebook:

1. **The sentence**: what the number measures, which rows it comes from, which rows the query left out
2. **The row estimate, with its source**: the brief, a section 0 cell, or "neither fixes this"
3. **The query, and the number**
4. **The check, in the kind the question names**: a sum, a ranked list, a ratio
5. **One line: how would I know if this were wrong?**

Then the **reconciliation** — by month = by country = the total, to the penny. Commits that name the cause · `GIT_LOG.txt` · the zip, as `SUBMITTING.md` says · `AI_USE.md`.